

资产管理系统 — 设计概要

文档版本：v1.0
编写日期：2026-06-10
文档性质：概要设计文档（High-Level Design Document）
关联文档：需求文档 v1.0、架构设计文档 v1.0

1. 系统架构总览

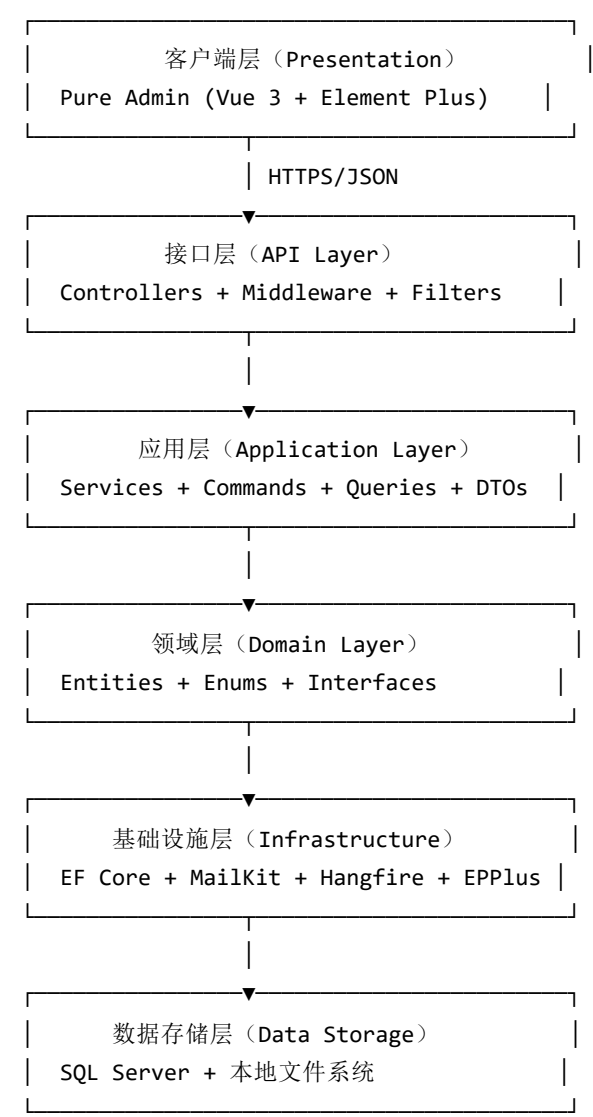
1.1 架构风格

- 整体架构：单体应用（Monolith）
- 前后端关系：分离（SPA + RESTful API）
- 部署模型：单服务器内网部署

1.2 技术选型

层次	技术栈	版本
前端框架	Vue 3 + TypeScript	3.4+
前端模板	Pure Admin	最新版
UI组件库	Element Plus	2.6+
状态管理	Pinia	2.1+
后端框架	ASP.NET Core Web API	8.0
ORM	Entity Framework Core	8.0
数据库	SQL Server	2017+
定时任务	Hangfire	1.8+
邮件发送	MailKit	4.4+
Excel处理	EPPlus	7.0+
身份认证	JWT	-

1.3 系统分层



2. 核心模块设计

2.1 资产档案模块

职责：资产信息的CRUD及状态管理

核心实体： Asset

字段: id, asset_no, name, category_id, model, brand, quantity, unit_price, supplier, location_id, custodian_id, status, attachment_paths, remark, is_deleted, created_at, updated_at

关键方法：

- CreateAsync() - 创建资产（含自动编号生成）
- UpdateAsync() - 更新资产信息
- ChangeStatusAsync() - 状态流转（含业务规则校验）
- ScrapAsync() - 逻辑删除

2.2 编号生成模块

职责：按类别规则生成唯一编号

核心实体： NumberRule

字段: id, category_id, segment_def (JSON),
seq_current, created_at, updated_at

segment_def 结构：

```
[  
  { "type": "fixed", "value": "PLC" },  
  { "type": "fixed", "value": "MIT" },  
  { "type": "seq", "length": 3, "start": 1 }  
]
```

并发控制：

- 使用乐观锁（UPDATE WHERE seq_current = @old）
- 失败重试最多3次，间隔50ms
- 编号生成与资产创建在同一事务内

2.3 审批流模块

职责：借还申请的审批流程管理

核心实体： ApprovalFlow

字段: id, flow_no, flow_type, asset_id, applicant_id,
approver_id, actual_approver_id, expected_return_date,
status, deadline, approve_time, reject_reason,
created_at, updated_at

状态机：

Pending（待审批）

- └─> Approved（已通过）
- └─> Rejected（已驳回）
- └─> AutoApproved（超时自动通过）

核心方法（领域层）：

- Approve() - 审批通过（含状态校验）
- Reject() - 审批驳回
- AutoApproveOnTimeout() - 超时自动通过

2.4 代审管理模块

职责：主管临时授权代审人

核心实体： ProxyApproval

字段： id, supervisor_id, proxy_user_id, start_date, end_date, is_active

逻辑：

- 发起申请时查询申请人的主管是否有有效代审配置
- 如有，审批人=代审人；否则，审批人=主管

2.5 定时任务模块

技术： Hangfire（In-Process）

任务列表：

任务名	Cron表达式	职责
超时自动审批扫描	0 */15 * * * *	每15分钟扫描待审批且已超时的工单，自动审批通过
逾期提醒	0 0 9 * * *	每天09:00扫描逾期未归还资产，发送提醒邮件
到期前提醒	0 0 9 * * *	每天09:00扫描即将到期资产，提前提醒
审计日志清理	0 0 2 1 * *	每月1日02:00清理超过保留期的审计日志

2.6 审计日志模块

职责：自动记录所有写操作

实现方式：AOP切面（ActionFilter）

核心实体： AuditLog

字段: id, user_id, action_type, target_type, target_id, summary, diff (JSON), ip_address, trace_id, occurred_at

记录策略：

- CREATE：diff=null，summary描述"新增资产 PLC-MIT-Q-004"
- UPDATE：diff记录字段变更前后值
- DELETE：diff=null，summary描述"删除资产 PLC-MIT-Q-004"
- STATUS_CHANGE：diff=null，summary描述"状态：在库→借出中"

写入方式：异步（Task.Run），不阻塞主请求

2.7 邮件通知模块

职责：发送各类业务通知邮件

技术：MailKit（SMTP）

邮件类型：

场景	收件人	触发时机
审批通知	审批人	申请提交后
审批结果通知	申请人	审批通过/驳回后
超时自动审批通知	申请人+审批人	超时自动通过后
逾期提醒	保管人+主管	每日09:00检测到逾期
到期前提醒	保管人	到期前N天

容错机制：

- 邮件发送失败写入审计日志（EMAIL_FAIL）
- 加入Hangfire重试队列，延迟5分钟重试
- 最多重试3次，失败后不再重试

2.8 导入导出模块

Excel导入（两阶段）：

1. 预校验阶段：
 - 解析Excel，逐行校验（类别/位置/必填项）
 - 生成sessionId，缓存校验结果（30分钟有效）
 - 返回校验报告
2. 确认导入阶段：
 - 根据sessionId读取有效行

- 批量写入（每100行一个事务）
- 生成导入报告

Excel导出：

- 基于当前筛选条件导出全部数据（不分页）
- 使用EPPlus生成.xlsx文件

3. 数据库设计要点

3.1 核心表结构

表名	说明	主要字段	索引
assets	资产档案	id, asset_no, name, category_id, location_id, status, custodian_id	asset_no(唯一), category_id, location_id, status, custodian_id
asset_categories	资产类别	id, name, parent_id, full_path	parent_id, full_path
number_rules	编号规则	id, category_id, segment_def, seq_current	category_id(唯一)
locations	存放位置	id, name, parent_id, full_path, qr_code	parent_id, full_path
users	用户	id, employee_no, name, role, department, supervisor_id, email, phone	employee_no(唯一), supervisor_id, role
approval_flows	审批工单	id, flow_no, flow_type, asset_id, applicant_id, approver_id, status, deadline	flow_no(唯一), asset_id, applicant_id, approver_id, status, deadline
flow_records	审批操作记录	id, flow_id, action, operator_id, comment, operated_at	flow_id, operated_at
proxy_approvals	代审配置	id, supervisor_id, proxy_user_id, start_date, end_date, is_active	supervisor_id, start_date, end_date, is_active
audit_logs	审计日志	id, user_id, action_type, target_type, target_id, summary, diff, occurred_at	user_id, action_type, target_type, occurred_at
system_settings	系统参数	id, key, value, description	key(唯一)

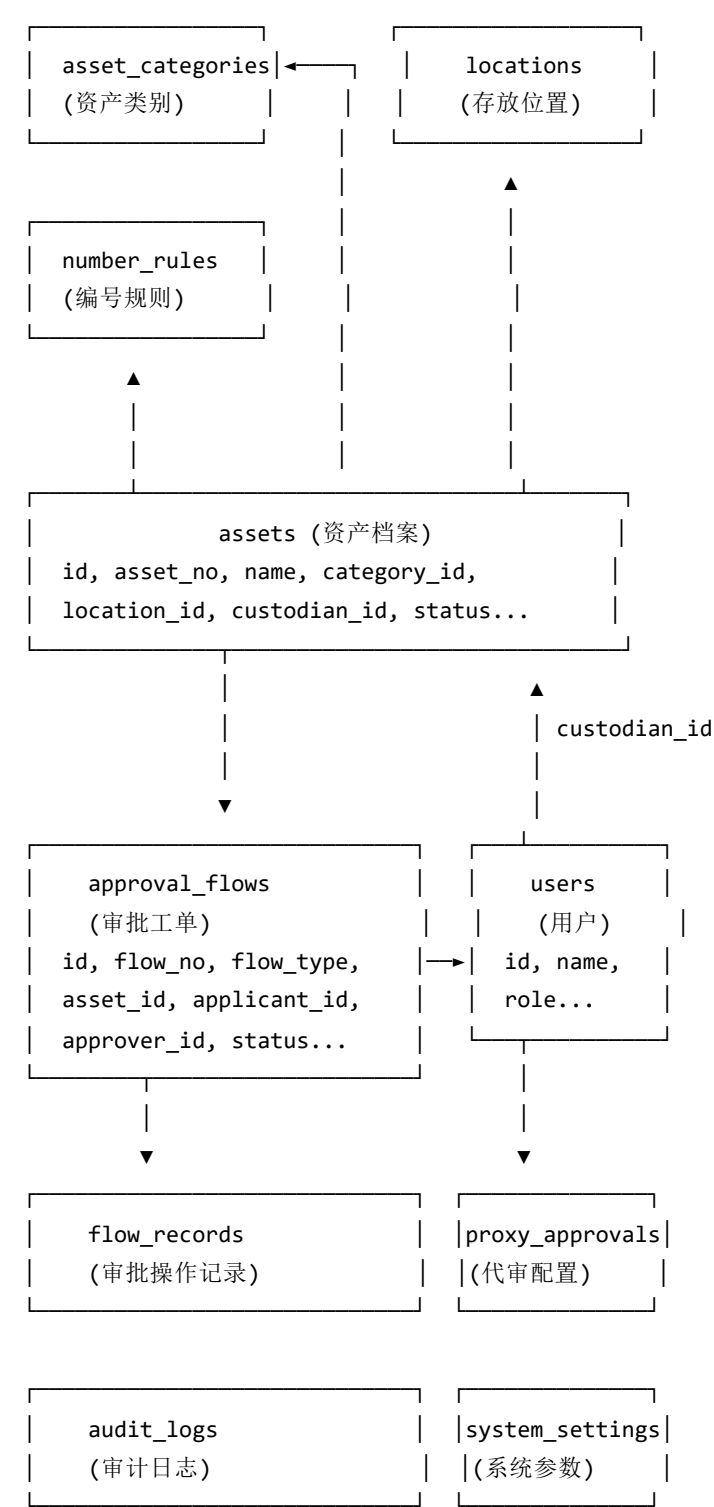
3.2 关键约束

- 唯一性约束：
 - `assets.asset_no` UNIQUE
 - `users.employee_no` UNIQUE
 - `approval_flows.flow_no` UNIQUE
 - `number_rules.category_id` UNIQUE
- 外键约束：
 - `assets.category_id` → `asset_categories.id`
 - `assets.location_id` → `locations.id`
 - `assets.custodian_id` → `users.id`
 - `approval_flows.asset_id` → `assets.id`
 - `approval_flows.applicant_id` → `users.id`
 - `approval_flows.approver_id` → `users.id`
- 检查约束：
 - `assets.status` IN ('available', 'borrowed', 'maintenance', 'scrapped')
 - `approval_flows.status` IN ('pending', 'approved', 'rejected', 'auto_approved')
 - `users.role` IN ('employee', 'warehouse', 'supervisor', 'admin')

3.3 索引策略

- 频繁查询字段: `status`, `category_id`, `location_id`, `custodian_id`
- 日期范围查询: `created_at`, `updated_at`, `occurred_at`, `deadline`
- 复合索引：
 - (`status`, `category_id`) - 按状态+类别筛选
 - (`applicant_id`, `status`, `created_at`) - 我的申请列表
 - (`approver_id`, `status`, `deadline`) - 待审批列表

3.4 数据库ER图



关系说明：

- assets.category_id → asset_categories.id (N:1)
- assets.location_id → [locations.id](#) (N:1)
- assets.custodian_id → [users.id](#) (N:1, 可空)
- approval_flows.asset_id → [assets.id](#) (N:1)

- approval_flows.applicant_id → [users.id](#) (N:1)
- approval_flows.approver_id → [users.id](#) (N:1)
- flow_records.flow_id → approval_flows.id (N:1)
- number_rules.category_id → asset_categories.id (1:1)
- proxy_approvals.supervisor_id → [users.id](#) (N:1)
- users.supervisor_id → [users.id](#) (N:1, 自关联)

4. 接口设计规范

4.1 RESTful API 设计原则

- **资源命名**：使用复数名词（ /assets , /approvals ）
- **HTTP方法**：GET（查询）、POST（创建）、PUT（完整更新）、PATCH（部分更新）、DELETE（删除）
- **状态码**：
 - 200 OK - 成功
 - 201 Created - 创建成功
 - 204 No Content - 删除成功
 - 400 Bad Request - 参数错误
 - 401 Unauthorized - 未认证
 - 403 Forbidden - 无权限
 - 404 Not Found - 资源不存在
 - 409 Conflict - 冲突（如编号重复）
 - 500 Internal Server Error - 服务器错误

4.2 统一响应格式

成功响应：

```
{
  "data": { ... },
  "message": "操作成功"
}
```

分页响应：

```
{
  "data": {
    "items": [ ... ],
    "total": 100,
    "page": 1,
    "pageSize": 20
  }
}
```

错误响应：

```
{
  "error": {
    "code": "ASSET_NOT_FOUND",
    "message": "资产不存在",
    "trace_id": "a1b2c3d4-e5f6-7890"
  }
}
```

4.3 统一错误码表

错误码	HTTP状态码	说明	示例场景
ASSET_NOT_FOUND	404	资产不存在	查询不存在的资产ID
ASSET_ALREADY_BORROWED	409	资产已被借出	借出中的资产再次发起借用申请
ASSET_CANNOT_SCRAP	400	资产不可报废	借出中的资产尝试报废
APPROVAL_NOT_FOUND	404	审批工单不存在	查询不存在的工单ID
APPROVAL_ALREADY_PROCESSED	409	审批已处理	重复审批同一工单
APPROVAL_TIMEOUT	409	审批已超时自动通过	超时后尝试手动审批
INVALID_APPROVER	403	无权审批此工单	非指定审批人尝试审批
NUMBER_GENERATE_FAILED	500	编号生成失败	乐观锁冲突3次重试后仍失败
CATEGORY_NOT_FOUND	404	类别不存在	新增资产时类别ID无效
CATEGORY_HAS_ASSETS	400	类别下有资产，不可删除	删除有关联资产的类别
LOCATION_NOT_FOUND	404	位置不存在	新增资产时位置ID无效
LOCATION_HAS_ASSETS	400	位置下有资产，不可删除	删除有关联资产的位置
USER_NOT_FOUND	404	用户不存在	查询不存在的用户ID
EMPLOYEE_NO_DUPLICATE	409	工号已存在	新增用户时工号重复
INVALID_CREDENTIALS	401	用户名或密码错误	登录失败
TOKEN_EXPIRED	401	Token已过期	过期Token访问接口
TOKEN_INVALID	401	Token无效	格式错误或被篡改
PERMISSION_DENIED	403	权限不足	普通员工访问管理员接口
VALIDATION_ERROR	400	参数校验失败	必填字段缺失、格式错误

错误码	HTTP状态码	说明	示例场景
FILE_TOO_LARGE	400	文件过大	上传超过20MB的文件
FILE_TYPE_NOT_ALLOWED	400	文件类型不支持	上传.exe等非法文件
IMPORT_SESSION_EXPIRED	400	导入会话已过期	预校验30分钟后提交确认
EMAIL_SEND_FAILED	500	邮件发送失败	SMTP服务不可用
DATABASE_ERROR	500	数据库错误	连接失败、超时
INTERNAL_ERROR	500	系统内部错误	未预期的异常

4.4 核心接口列表

资产管理

- GET /api/assets - 资产列表（分页+筛选）
- GET /api/assets/{id} - 资产详情
- POST /api/assets - 新增资产
- PUT /api/assets/{id} - 更新资产
- DELETE /api/assets/{id} - 报废资产
- GET /api/assets/{id}/attachments/{filename} - 下载附件

审批流

- GET /api/approvals - 审批列表
- GET /api/approvals/{id} - 审批详情
- POST /api/approvals - 发起申请
- POST /api/approvals/{id}/approve - 审批通过
- POST /api/approvals/{id}/reject - 审批驳回
- POST /api/approvals/{id}/confirm-return - 确认归还入库

导入导出

- POST /api/import/validate - Excel预校验
- POST /api/import/confirm - 确认导入
- GET /api/import/template - 下载模板
- GET /api/export/assets - 导出资产列表

统计报表

- GET /api/reports/summary - 资产汇总报表
- GET /api/reports/borrowed - 借用明细报表
- GET /api/reports/overdue - 逾期资产报表

系统管理

- GET /api/users - 用户列表
- POST /api/users - 新增用户
- PUT /api/users/{id} - 编辑用户
- GET /api/categories - 类别树
- GET /api/locations - 位置树
- GET /api/settings - 系统参数
- PUT /api/settings - 更新参数
- GET /api/audit-logs - 审计日志

5. 安全设计

5.1 认证机制

- **JWT Token**: Payload含 {sub, name, role, exp}
- **有效期**: 8小时
- **续签策略**: 剩余<1小时时响应头携带新Token
- **注销**: Token加入黑名单（存数据库或Redis）

5.2 权限控制

- **接口级权限**: 使用 [RequireRole("admin")] 特性
- **数据级权限**:
 - 普通员工只能查看自己的申请
 - 主管可查看所有下属申请
 - 管理员查看全部

5.3 输入验证

- **参数校验**: 使用FluentValidation
- **SQL注入防护**: 全程使用EF Core参数化查询
- **XSS防护**: 前端Vue模板自动转义, 后端不渲染HTML
- **路径遍历防护**: 文件名使用GUID, 不使用用户上传原始名

5.4 日志与监控

- **操作日志**: 所有写操作自动记录（含trace_id）
- **错误日志**: 异常全局捕获, 记录堆栈和上下文
- **性能日志**: 记录慢查询（>1秒）

6. 部署架构

6.1 单服务器部署拓扑

```
Windows Server (内网)
├─ IIS
│   ├── 静态站点 (Vue SPA) :80
│   └─ API应用程序 (反向代理到 Kestrel:5000)
├─ ASP.NET Core 进程
│   ├── Web API
│   └─ Hangfire Server
├─ SQL Server :1433
└─ 本地文件系统 (附件存储)
```

6.2 发布流程

前端：

```
npm run build
# 将 dist/ 部署到 IIS 静态站点
```

后端：

```
dotnet publish -c Release -r win-x64 --self-contained -o C:\AssetMgmt\app
# 配置 IIS 应用程序池（无托管代码）
# 配置 web.config (AspNetCoreModuleV2)
```

数据库迁移：

```
dotnet ef database update
# 或启动时自动迁移：--migrate 参数
```

6.3 健康检查

接口：GET /health

检查项：

- 数据库连接 (SQL Server)
- Hangfire服务状态
- 磁盘空间 (附件存储路径)

6.4 缓存策略

需要缓存的数据

数据类型	缓存位置	失效策略	原因
类别树	前端Pinia	刷新页面时重新加载	类别数据稳定，变更频率低
位置树	前端Pinia	刷新页面时重新加载	位置数据稳定，变更频率低
系统参数	后端MemoryCache	管理员修改后失效	避免频繁查询数据库
当前用户信息	前端Pinia	Token过期时清空	减少重复查询

不缓存的数据

- 资产列表（实时性要求高）
- 审批工单列表（状态变化频繁）
- 审计日志（实时追溯需求）
- 统计报表（数据实时统计）

6.5 性能优化策略

数据库层面

- **N+1查询预防**：使用EF Core的 `.Include()` 预加载关联数据
- **批量操作**：导入时每100行一个事务，减少提交次数
- **分页查询**：列表查询强制分页，避免一次加载过多数据
- **索引优化**：对高频查询字段建立索引（见3.3节）

应用层面

- **异步处理**：审计日志、邮件发送采用异步，不阻塞主流程
- **并发控制**：编号生成采用乐观锁，减少锁等待时间
- **连接池**：配置数据库连接池（最小5，最大50）

前端层面

- **懒加载**：类别树、位置树首次访问时加载，缓存到Pinia
- **防抖节流**：搜索输入框300ms防抖，避免频繁请求
- **虚拟滚动**：报表预览超过500行时使用虚拟滚动

6.6 前端路由设计

路由路径	组件	权限	说明
/login	LoginView	公开	登录页
/	LayoutView	所有角色	主布局（含导航）

路由路径	组件	权限	说明
/assets	AssetListView	所有角色	资产列表
/assets/:id	AssetDetailView	所有角色	资产详情
/assets/new	AssetFormView	warehouse, admin	新增资产
/assets/:id/edit	AssetFormView	warehouse, admin	编辑资产
/assets/import	AssetImportView	warehouse, admin	批量导入
/approvals/pending	PendingView	supervisor, admin	待我审批
/approvals/mine	MineView	所有角色	我的申请
/approvals/completed	CompletedView	warehouse, supervisor, admin	已完成审批
/approvals/:id	ApprovalDetailView	相关用户	审批详情
/reports/summary	SummaryView	warehouse, supervisor, admin	资产汇总
/reports/borrowed	BorrowedView	warehouse, supervisor, admin	借用明细
/reports/overdue	OverdueView	warehouse, supervisor, admin	逾期资产
/admin/users	UserManageView	admin	用户管理
/admin/categories	CategoryManageView	warehouse, admin	类别管理
/admin/locations	LocationManageView	warehouse, admin	位置管理
/admin/settings	SettingsView	admin	系统参数
/admin/audit-logs	AuditLogView	warehouse, admin	审计日志
/403	ForbiddenView	公开	无权限页
/404	NotFoundView	公开	页面不存在

路由守卫逻辑：

```

router.beforeEach((to) => {
  const authStore = useAuthStore()

  // 未登录 → 跳转登录页
  if (!authStore.token && to.path !== '/login') {
    return '/login'
  }

  // 权限校验
  const requiredRoles = to.meta.roles as string[]
  if (requiredRoles && !requiredRoles.includes(authStore.user.role)) {
    return '/403'
  }
})

```

7. 关键技术要点

7.1 编号生成并发控制

```

-- 乐观锁更新
UPDATE number_rules
SET seq_current = seq_current + 1, updated_at = GETDATE()
WHERE id = @id AND seq_current = @expected_seq;

-- 检查受影响行数，若为0则重试

```

7.2 审计日志异步写入

```

// ActionFilter 中异步写入，不阻塞主请求
await Task.Run(() => _auditLogService.LogAsync(auditLog));

```

7.3 邮件发送容错

```

try {
  await _mailKitService.SendAsync(message);
} catch (Exception ex) {
  await _auditLogService.LogEmailFailAsync(ex);
  BackgroundJob.Schedule(() => RetryEmail(message), TimeSpan.FromMinutes(5));
}

```


7.4 Excel导入校验缓存

```
// 预校验后生成sessionId, 结果缓存30分钟
var sessionId = Guid.NewGuid().ToString();
await _cache.SetAsync(sessionId, validationResult, TimeSpan.FromMinutes(30));

// 确认导入时读取缓存
var result = await _cache.GetAsync<ValidationResult>(sessionId);
```

8. 开发规范

8.1 命名约定

- 数据库表名：小写+下划线（ asset_categories ）
- C# 类名：Pascal命名（ AssetCategory ）
- C# 方法名：Pascal命名（ CreateAssetAsync ）
- 接口路由：小写+短横线（ /api/asset-categories ）
- 前端组件：Pascal命名（ AssetListView.vue ）
- 前端方法：camel命名（ fetchAssetList ）

8.2 异步规范

- 所有I/O操作使用异步方法（ async/await ）
- 异步方法命名后缀 Async
- 避免 async void （除事件处理器）

8.3 异常处理

- Controller层不捕获异常（交给全局中间件）
- Service层抛出业务异常（ DomainException ）
- 基础设施层捕获外部异常并转换为领域异常

文档版本历史：

- v1.0 (2026-06-10)：初始版本，涵盖系统架构、模块设计、接口规范